Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Multi-Lane Parking Allocation and Rerouting Simulator

Multi-Lane Parking Allocation and Rerouting Simulator

Student / Team Information

Team Name: *Team #DSCPP-III-2026-T062*	*The Canvas*
Team member 1 (Team Lead)	*Bhatt, Aayush – 2510011035* *2510011035@geu.ac.in* 
Team member 2	*Sachdeva, Armaan – 2510012259* *2510012259@geu.ac.in* 
Team member 3	*Jain, Arihant – 2510011114* *2510011114@geu.ac.in* 
Team member 4	*Aithani, Aman – 25101280977* *25101280977@geu.ac.in* 

PHASE-II PROJECT PROGRESS DESCRIPTION

Project Abstract

Around 30% of downtown traffic comes from drivers searching for parking, which wastes fuel and spikes carbon emissions. This project builds a smart parking framework that guides cars through a crowded multi-level garage to the nearest suitable spot without unnecessary driving. The end goal is to cut down search time and lower emissions caused by cruising for open spots.

To address this, we built an event-driven simulation in C++. The model parses a custom layout file that defines entry/exit gates, lanes, and spot types. Events—like vehicle arrivals, departures, or temporary lane blocks—are handled in strict order using a priority queue. When a vehicle arrives, the system checks its size, fuel type, and accessibility needs to assign a spot.

For routing, we run an A* pathfinding algorithm with dynamic weights to steer drivers away from busy lanes. Reservations use interval trees to prevent overlapping time slots. The system also tracks parking duration to handle billing and monitor overall revenue.

Finally, we built a terminal UI using ncurses/PDCurses to show real-time vehicle movement and spot availability.

Updated Project Approach and Architecture

Language and tools: Written entirely in C++ and compiled with g++. The code is split into clean modules (event queue, lane graph, pathfinding, reservations, models, ticketing/metrics, visualizer, orchestrator), each with dedicated header and source files.

Data structures and algorithms: We built the lane graph and interval tree from scratch. The lane graph uses an adjacency list where dynamic weights rise as lane occupancy increases. The interval tree flags conflicting time slots and rejects bad bookings. Priority queues handle both event ordering and routing (Dijkstra/A*). Core logic uses standard C++ vector types with no external dependencies so far.

Working: The engine processes events from the event queue in the order they occur: vehicle The system takes into account arrivals, exits, and lane closures; when a vehicle enters it is given an appropriate slot according to reach that slot, the vehicle uses the lane graph to find a route and then saves the booking and ticket. Since lane occupancy changes as vehicles move, the system keeps checking it and reroutes vehicles around lanes Those that are crowded or closed will prevent anything from getting stuck at a bottleneck.

Libraries: cmath is used for formula and calculations, chrono.h for time calculation and utilization. The terminal visualizer will use ncurses (for Linux) / PDCurses(for Windows).

Tasks Completed

Task Completed	Team Member
Event queue (min-heap ordered by time, any conflicts looked onto by arrival order) with unit test	Armaan Sachdeva
Lane graph with congestion based weights, lane capacities and lane closures, with unit tests	Armaan Sachdeva
Interval tree for conflict free, time window reservations, with unit tests	Armaan Sachdeva
Dijkstra and A* pathfinding over the lane graph	Arihant Jain
Vehicle class hierarchy (standard, electric and accessible vehicles)	Aman Aithani
Parking slot class with slot states (free, occupied, reserved, maintenance)	Aman Aithani
Parking tickets and admin metrics (total revenue, occupancy rate, dashboard)	Aman Aithani
JSON layout loader for custom parking layouts	Aayush Bhatt
Collected references and background studies on the flaws in the current parking systems	Aayush Bhatt

Challenges/Roadblocks

Our modules were built in parallel by different people, so most of our problems came from making them fit together.

1. The vehicle model stores vehicle IDs as strings, while our event queue uses integers. We are deciding upon which one should be preferred over the other.

2. Our pathfinding returns a route cost that includes congestion, but billing and emissions need the real distance driven. We are adding separate base-distance functions to the graph so each part gets the number it actually needs.

3. While testing the pathfinder we found that the distance calculation crashes when no route exists (an empty path). The fix is a one-line check, and we are applying it.

4. To report emissions saved we must define a baseline, such as the route a vehicle would take with no congestion-aware routing, and pick an emission factor for each vehicle type. We are agreeing on these values as a team and will document the source.

5. Some of the libraries (such as PDcurses) were Windows exclusive which was causing the project to have issues running on UNIX based system. We are trying to made the same functionality to Linux using some equivalent libraries or trying to find libraries which has more cross platform connectivity.

We are working in a shared repository for the first time. Some of us were new to Git, so we adopted a simple workflow: one branch per module, pull requests, and merging into main only after tests pass. Our biggest remaining risk is integration. To reduce it, we are building a single orchestrator that runs the full event loop and connects every module, and we will test it with a small sample layout first. Adapting to this new setup felt harder than we had expected.

Tasks Pending

Task Pending	Team Member (to complete the task)
Live terminal visualizer (traffic, occupancy, revenue, emissions), work in progress	Aayush Bhatt
Orchestrator and `main()` that runs the event loop and reroutes vehicles when a lane is closed due to filled spots or traffic	Aayush Bhatt
Build setup (Makefile), README and documentation	All Members
Slot allocation logic and fee calculation depending on the parking duration and vehicle type	Aman Aithani
Trip metrics (time to park, parking duration, emissions saved)	Armaan Sachdeva
Real distance functions in the lane graph	Armaan Sachdeva
Empty path fix and unit tests for pathfinding	Arihant Jain
Final tests on a full sample layout	All members

Project Outcome/Deliverables

1. A terminal based simulation engine for entry and exit of vehicle, parking slots, and lane closures inside the parking facility, using real time path recalculation and rerouting.

2. A navigation module that finds the fastest route to a vehicle's assigned slot or to the exit while avoiding blocked or crowded lanes using algorithms such as A*.

3. A live and interactive terminal visualizer made using ncurses that shows traffic, occupancy,revenue generated by the establishment and estimated emissions saved by the vehicles by using the shorter path.

4. An allocation and billing module that matches vehicle needs to dedicated zones and calculates fees and revenue from parking duration.

5. A reservation module that books exact time windows without conflicts by reserving a certain spot to a certain vehicle.

6. A configuration loader for custom layouts using JSON, which allows us to test different types of parking layout, supported by tests and documentation.

7. Administrator reports containing revenue, occupancy rate, and the parking ticket history

8. Added billing and records fees by vehicle category & size, a ticket for every stay, & a tickets.csv export for the administrator

Progress Overview

We are estimating that the majority of the project is completed. The core algorithms are done and tested on their own, which include the event queue, the lane graph with dynamic weights, the interval tree for reservations, and the pathfinding. The vehicle, slot, ticket and admin metrics models have also been correctly placed and custom parking layouts can now be loaded from JSON files. We are ahead of schedule on these core parts.

We are behind on integration and the parts that the user would see. There is no orchestrator yet, so the modules have not been able to run together. The fee calculation and the terminal visualizer are also yet to be done. The task with the highest priority for us in the remaining time is the orchestrator, because every other pending part depends on it, directly or indirectly as it integrates all the module to deliver the final project. After that, we will finish the visualizer, complete the documentation and run a full test on a sample layout.

Codebase Information

Repository: https://github.com/iamaayushbhatt/PBL-parking-allocation
Branch: main

Important commits:

- Event structure and EventQueue class (078f066, 1156e03)
- Graph with dynamic lane weights, capacity and lane closure (f0bdf97)
- Interval tree header, madeation and tests (2626e02, 246d56b)
- Pathfinding with Dijkstra and A* (4f353fc)
- Vehicle class hierarchy (923607f)
- Parking tickets and admin metrics (644acb7)

Pull requests #1 and #2 include the module work that was merged. Tests are in the tests/ folder.

Testing and Validation Status

Test Type	**Status (Pass/Fail)**	**Notes**
Event queue test	Pass	Events return in correct order, and matching arrival times preserve the original sequence .
Lane graph test	Pass	Successfully handles capacity changes , dynamic lane weights and closures.
Interval tree test	Pass	Accurately flags overlapping bookings while allowing adjacent ones, and removal functions properly.
Pathfinding (Dijkstra and A*)	Partially tested	Compiles with the lane graph and finds routes on a manual check. Automated tests and a fix for when the path is empty are still pending.
Slots, tickets and admin metrics	Pending	The code builds fine, but we still need to write the test cases.
Final test	Pending	Waiting on the orchestrator and visualizer to link all the separate modules together.

Deliverables Progress

Majority of the project's objectives are achieved and some are still on going, here is a list of milestones that are completed and are in progress:

Completed:

1. Navigation: Vehicles are navigated using A* and Dijkstra on crowded lanes and are rerouting to emptier lanes.
2. JSON parser: is used for adding custom layouts for different types of parking spaces.
3. Time based reservation: Slots are pre-booked by drivers
4. Vehicle specific zones: Zones for vehicle are divided based on fuel type(EV/petrol) and accessibility spots are available for disabled people

In Progress:

1. Simulation Engine: Event queue is completed, an orchestrator that would run them together is pending.
2. Administrator reports: Record of vehicles for future reference is stored.
3. Live visualizer: An interactive visualizer using ncurses/PDcurses is to be made.
4. Tests and documentation: a basic Read-Me is made, documentation for each individual part is remaining.
5. Billing: The fee calculation module is still pending.